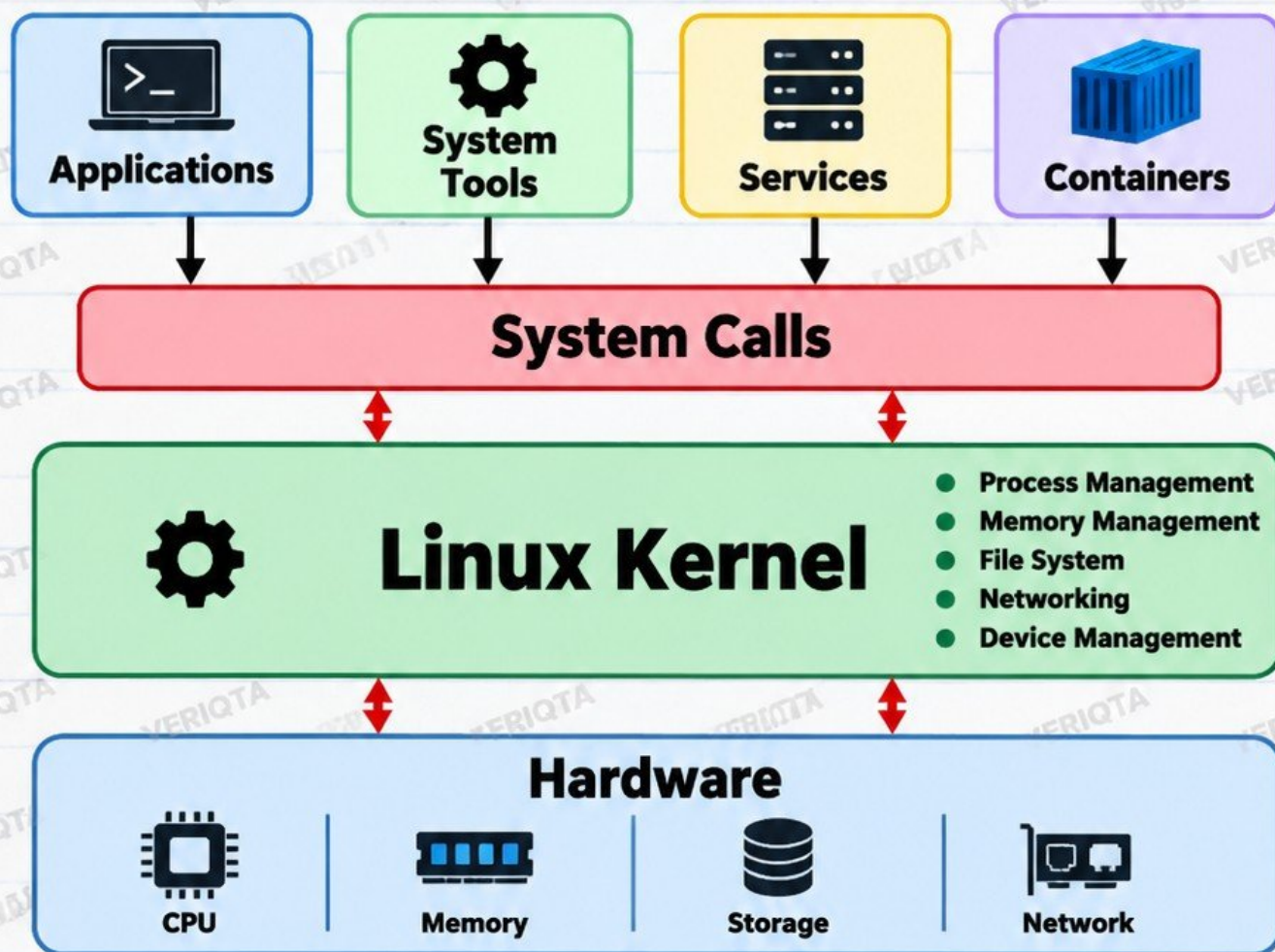


Linux Internals

for DevOps Engineers

Understand how Linux works, investigate problems, and build real DevOps skills.



Clear explanations



Real commands and examples



Troubleshooting guides



Production insights



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Inside a Linux System

The big picture: hardware → kernel → system calls → processes → userspace applications. Establish the mental model for everything that follows.

1. WHAT IS LINUX?

- Linux is an operating system that manages your computer hardware and runs applications.
- It provides a kernel, system tools and a user space environment.
- You interact with Linux using a shell, graphical interface or applications.

2. WHY DOES IT EXIST?

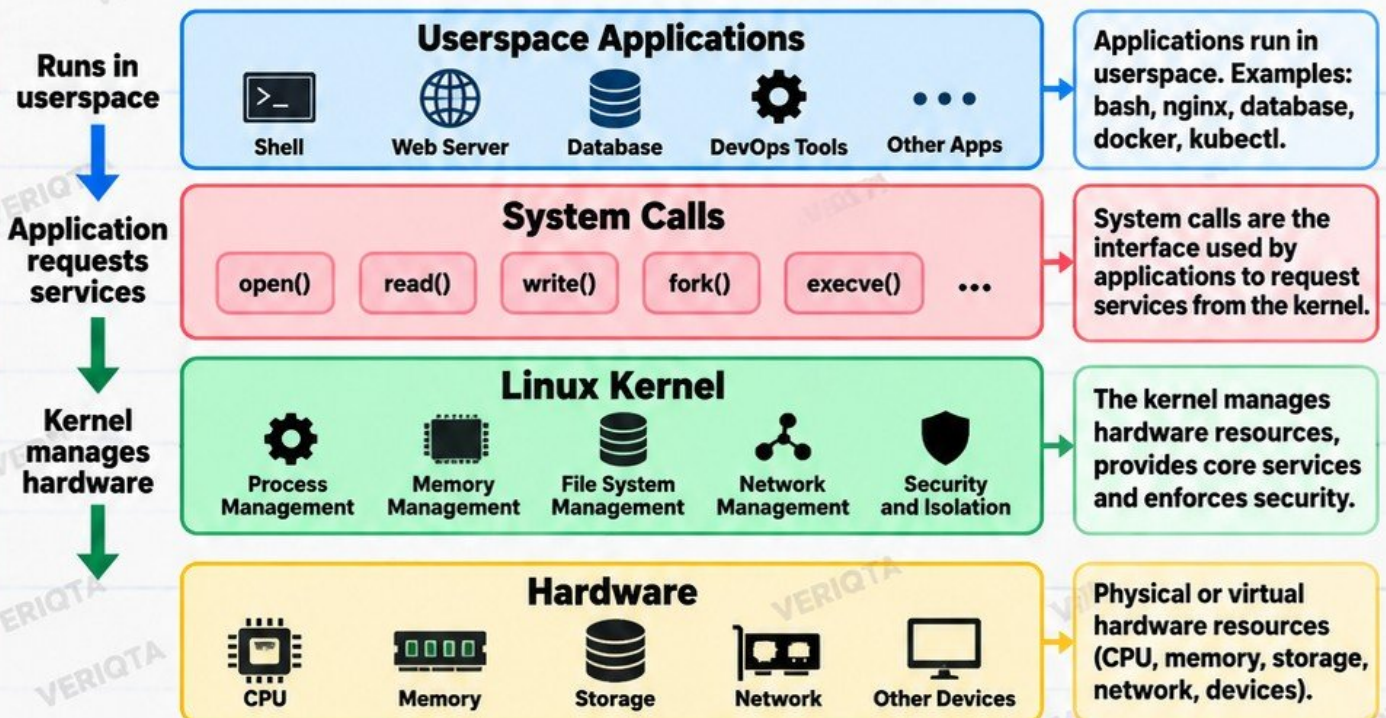
- To allow multiple applications to share hardware safely.
- To manage system resources (CPU, memory, storage, network).
- To provide security, isolation and stability.
- To give a standard environment for running applications and services.

3. REAL DEVOPS CONTEXT

- Linux runs servers, containers, cloud instances and Kubernetes nodes.
- DevOps engineers manage Linux systems, troubleshoot issues and automate operations.
- Understanding how Linux works helps you diagnose problems faster and make better decisions in production.

4. KEY COMPONENTS

- **Hardware** – physical or virtual resources.
- **Kernel** – core of Linux that manages hardware.
- **System calls** – interface between applications and the kernel.
- **Processes** – running programs.
- **Userspace applications** – tools, services and applications you use.



KEY TAKEAWAY

Applications do not talk directly to hardware. They use system calls to request services from the kernel. The kernel manages the hardware and provides resources in a safe and controlled way.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Kernel vs User Space



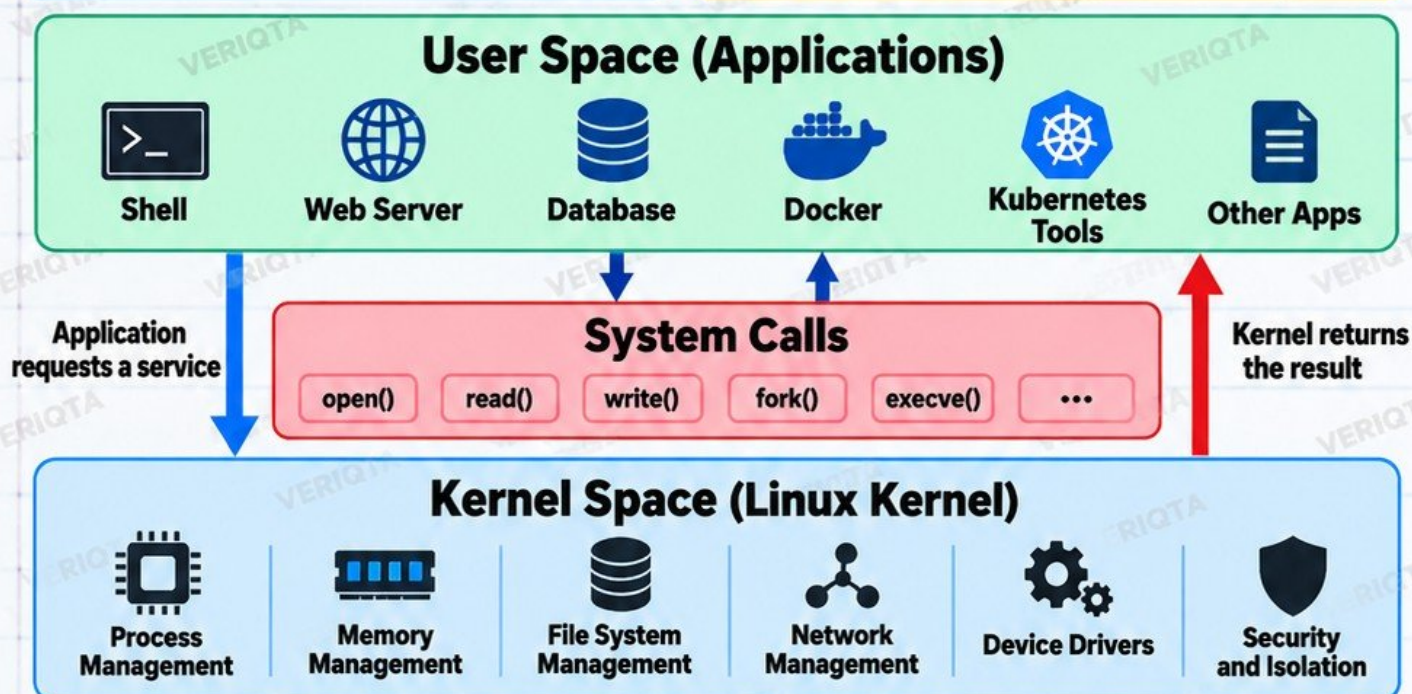
Why Linux separates privileged kernel execution from applications, what each side can access, and why this boundary matters for security and failures.

1. WHAT IS USER SPACE?

- Where your applications and system tools run.
- Does not have direct access to hardware.
- Uses system calls to request services from the kernel.
- Examples: shell, web server, database, Docker, kubectl, editors.

2. WHAT IS KERNEL SPACE?

- The core of Linux (the kernel).
- Runs with high privileges.
- Manages hardware and system resources.
- Provides services to applications through system calls.
- Examples: process scheduler, memory manager, file system, network stack, device drivers.



3. WHY SEPARATE THEM?

- Protects the system from bugs in applications.
- Prevents applications from directly accessing hardware.
- Provides a stable and controlled interface (system calls).
- Improves security, isolation and stability.

4. WHAT CAN EACH ACCESS?

	User Space	Kernel Space
Run applications	✓	✗
Access hardware directly	✗	✓
Manage processes	✗	✓
Manage memory	✗	✓
Access file system	Through system calls	✓
Load device drivers	✗	✓

5. REAL WORLD EXAMPLE

- A web server (user space) cannot read data from disk directly.
- It calls a system call like `read()`.
- The kernel checks permissions, reads the data and returns the result.

6. KEY TAKEAWAY



Applications do not talk to hardware directly. They use system calls to request services from the kernel. This separation keeps Linux secure, stable and reliable.

7. COMMON FAILURE



If the kernel or a kernel component fails, it can affect the entire system. Application crashes usually do not bring down the whole system because they run in user space.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

System Calls



How applications request kernel services. Introduce `open()`, `read()`, `write()`, `fork()`, `execve()` and why tools such as `strace` are useful.

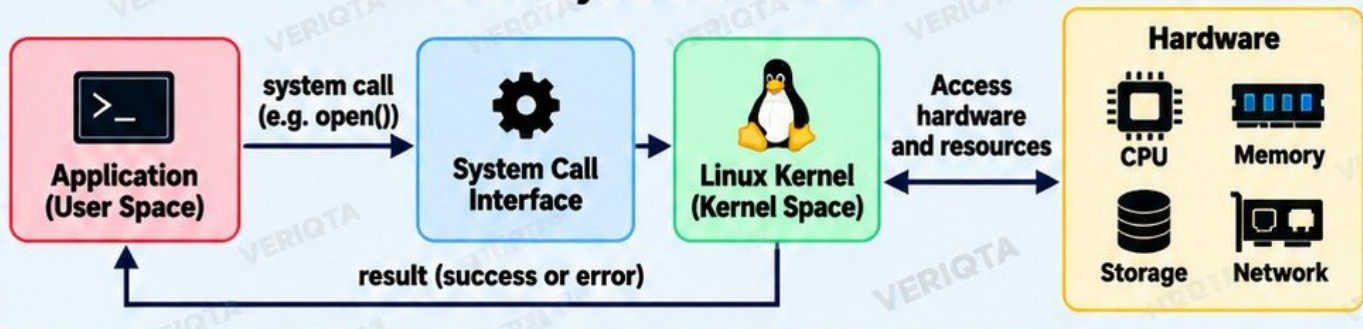
1. WHAT ARE SYSTEM CALLS?

- System calls are the interface used by applications to request services from the Linux kernel.
- Applications cannot access hardware directly.
- They use system calls to ask the kernel to perform tasks such as reading a file, creating a process or accessing the network.

2. HOW IT WORKS

- Application calls a library function (e.g. `open()`).
- The library makes a system call into the kernel.
- The kernel performs the requested operation.
- The kernel returns a result to the application.

How a System Call Flows



3. COMMON SYSTEM CALLS

System Call	What it Does	When You Use It	Simple Example
<code>open()</code>	Opens a file and returns a file descriptor.	When an application needs to read or write a file.	<code>open("/etc/passwd", O_RDONLY)</code>
<code>read()</code>	Reads data from a file descriptor.	When an application reads data from a file or device.	<code>read(fd, buffer, size)</code>
<code>write()</code>	Writes data to a file descriptor.	When an application writes data to a file or device.	<code>write(fd, buffer, size)</code>
<code>fork()</code>	Creates a new process by duplicating the current one.	Used to create a child process.	<code>pid = fork()</code>
<code>execve()</code>	Replaces the current process image with a new program.	Used after <code>fork()</code> to run a different program.	<code>execve("/bin/ls", argv, envp)</code>

4. WHY SYSTEM CALLS MATTER

- Provide a controlled and secure way for applications to use kernel services.
- Enforce permissions and isolation.
- Prevent applications from directly accessing hardware.
- Allow Linux to manage resources efficiently.

5. USING `strace`

- `strace` shows the system calls made by a program.
- Useful to understand what an application is doing.
- Helps with troubleshooting and debugging.

Example: trace a command
`strace ls`

6. KEY TAKEAWAY



Applications use system calls to request services from the kernel. Tools like `strace` help you see these calls and understand what an application is doing under the hood.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Processes Under the Hood

PID, PPID, process state, credentials, memory, file descriptors and the process lifecycle. Connect this to services and containers.

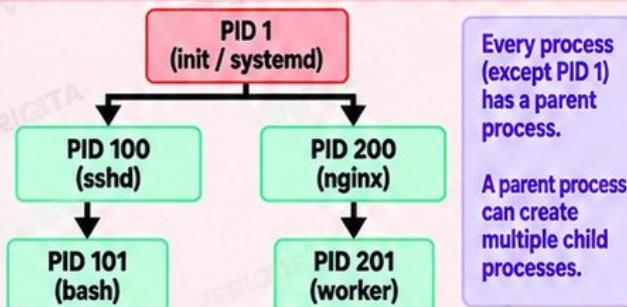
1. WHAT IS A PROCESS?

- A process is a running program in memory.
- It has a unique Process ID (PID).
- It uses system resources such as CPU, memory, files and network.
- Every process is created by another process (parent process).

2. IMPORTANT PROCESS INFORMATION

PID	Unique identifier for the process.
PPID	ID of the parent process.
User / Group	Runs with specific user and group permissions.
State	Current status (running, sleeping, etc).
Memory	Amount of memory the process is using.
File descriptors	Open files, sockets, pipes and devices used by the process.
Command	The program or command being executed.

3. PROCESS TREE (PARENT AND CHILD)



4. PROCESS STATES

State	Meaning
R (running)	Currently using CPU.
S (sleeping)	Waiting for an event (e.g. I/O, timer).
D (uninterruptible)	Waiting for I/O and cannot be interrupted.
T (stopped)	Paused (e.g. by a signal).
Z (zombie)	Finished execution but still in process table waiting for parent to collect exit status.

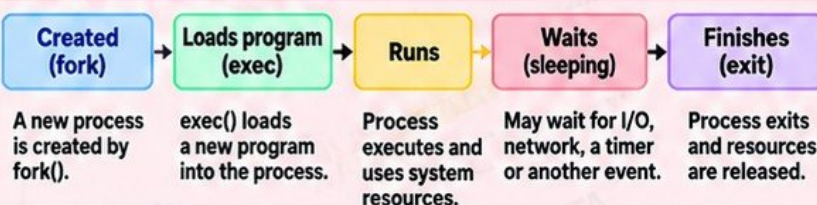
5. VIEWING PROCESSES

ps aux	List all running processes.
ps -ef	List processes in full format.
top	Real-time view of running processes.
htop	Interactive process viewer (may need to install).
ps tree -p	Show process tree with PIDs.

6. EXAMPLE OUTPUT (ps aux)

USER	PID	PPID	%CPU	%MEM	STAT	COMMAND
root	1	0	0.0	0.1	Ss	/sbin/init
root	100	1	0.0	0.2	Ss	/usr/sbin/sshd
veriqta	101	100	0.0	0.1	S	-bash
root	200	1	0.1	0.5	S	nginx: master
www	201	200	0.0	0.3	S	nginx: worker

7. PROCESS LIFECYCLE



8. REAL DEVOPS CONTEXT

- Services: systemd starts processes (e.g. nginx, sshd) and keeps them running.
- Containers: each container runs one or more processes. The main process inside a container is PID 1 within that container.
- Monitoring: processes consume CPU, memory, files and network. Understanding processes helps you troubleshoot performance issues.
- Debugging: you can inspect, pause or terminate processes when something goes wrong.

9. COMMON ISSUES

- Process using high CPU or memory.
- Zombie processes (Z state).
- Too many open files (file descriptors).
- Stuck processes in uninterruptible sleep (D state).

10. KEY TAKEAWAY



A process is a running program with a PID, parent, state, permissions, memory and open files. Understanding processes is essential for managing services, containers and troubleshooting Linux systems.

11. TRY THIS



1. Run `ps aux` and identify a process of your choice.
2. Find its parent process (PPID).
3. Check its memory usage.
4. See its open files:
`ls -l /proc/<PID>/fd`



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Process Creation: fork + exec

How Linux creates processes, what `fork()` conceptually does, what `exec()` changes, parent/child relationships and practical process trees.

1. WHAT IS `fork()`?

- `fork()` creates a new process by copying the current process.
- The new process is a child process with a different PID.
- The child gets a copy of the parent's memory, file descriptors and environment.
- Both parent and child continue running separately.

2. WHAT IS `exec()`?

- `exec()` replaces the current process image with a new program.
- It does not create a new process.
- The PID remains the same after `exec()`.
- The new program gets a fresh memory space.
- It inherits open file descriptors unless they are marked to close.

3. HOW `fork()` AND `exec()` WORK TOGETHER

Parent Process
(PID 100)



Running a shell
(e.g. bash)

`fork()`

Child Process
(PID 200)



Copy of parent process

`exec()`

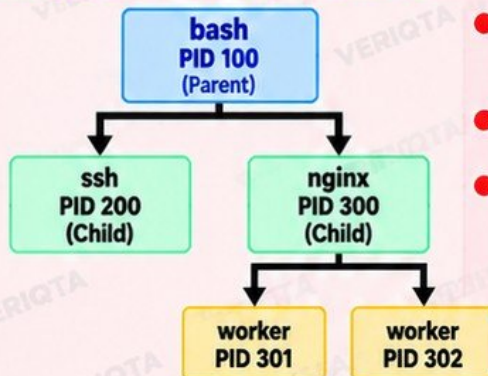
Child Process
(PID 200)



New program
(e.g. ls)

- `fork()` creates a new process (that is a copy of the parent).
- `exec()` replaces the child's memory with a new program.
- This is how a shell runs commands.
- The parent (shell) usually waits for the child to finish.

4. PROCESS TREE EXAMPLE



- A parent process can create multiple child processes.
- Each process has a unique PID.
- Processes can create their own child processes, forming a tree.

5. PRACTICAL EXAMPLE

```
# Start a process in background
sleep 100 &
# This creates a new process using fork + exec
```

```
# View the process tree
pstree -p | head
# Shows parent and child relationships
```

```
# View a process and its children
ps -f --forest -p 1
# Show process tree for PID 1 (init/systemd)
```

6. KEY POINTS

- `fork()` creates a new process (copy).
- `exec()` replaces the process image.
- The child has a new PID after `fork()`.
- The PID stays the same after `exec()`.
- The parent and child run independently.

7. REAL DEVOPS CONTEXT

- Web servers (e.g. nginx) use a master process to create worker processes.
- Process managers (e.g. systemd) start services using `fork + exec`.
- Containers also use `fork + exec` to start the main process (PID 1).
- Understanding this helps you debug services, containers and background processes.

8. TRY THIS

- 1 Run `sleep 200 &`
- 2 Use `ps -f` to find the PID.
- 3 Check its parent PID (PPID).
- 4 Use `pstree -p` to view the process tree.
- 5 Kill the process and check how the tree changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Linux Process States



Running, runnable, sleeping, uninterruptible sleep, stopped and zombie processes. Show why a process being “alive” does not mean it is healthy.

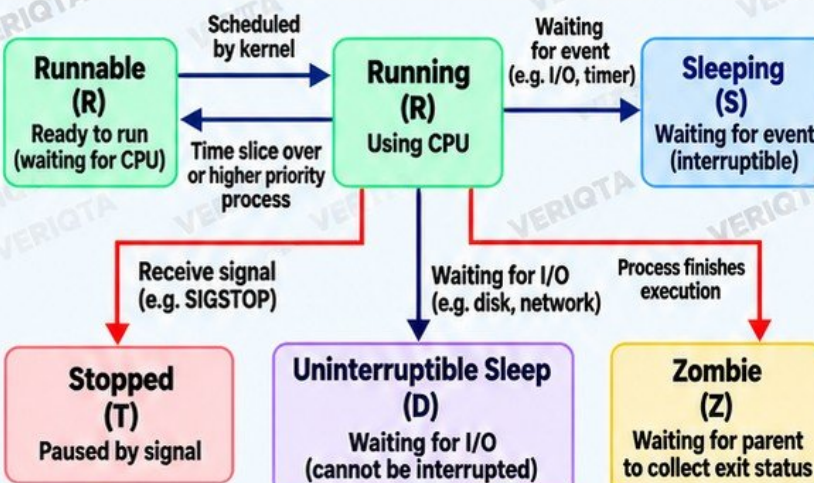
1. WHAT ARE PROCESS STATES?

- A process can be in different states based on what it is doing.
- The state shows whether it is running, waiting for resources or stopped.
- A process can be “alive” but not making progress.
- Understanding process states helps you troubleshoot performance issues and system problems.

2. COMMON PROCESS STATES

- Running (R) – currently using CPU.
- Runnable (R) – ready to run, waiting for CPU.
- Sleeping (S) – waiting for an event (e.g. I/O).
- Uninterruptible sleep (D) – waiting for I/O, cannot be interrupted.
- Stopped (T) – paused by a signal.
- Zombie (Z) – finished execution, waiting for parent to collect exit status.

3. PROCESS STATE DIAGRAM



4. HOW TO CHECK PROCESS STATE

<code>ps aux</code>	Shows all processes and their states (STAT column).
<code>ps -ef</code>	Full format view of processes.
<code>top</code>	Interactive view of running processes.
<code>htop</code>	Enhanced interactive view (if installed).
<code>ps tree -p</code>	Shows process tree with PIDs and states.

5. STATE CODES IN ps

Code	Meaning
R	Running (using CPU)
R	Runnable (ready to run)
S	Sleeping (interruptible)
D	Uninterruptible sleep (I/O)
T	Stopped (by signal)
Z	Zombie (defunct)

6. EXAMPLE OUTPUT (ps aux)

USER	PID	PPID	%CPU	%MEM	STAT	COMMAND
root	1	0	0.0	0.1	Ss	/sbin/init
root	234	1	0.0	0.2	S	nginx: master
www	235	234	0.0	0.3	S	nginx: worker
veriqta	560	540	10.2	1.1	R	python app.py
root	892	1	0.0	0.1	D	[kworker/0:1]
root	1044	1	0.0	0.0	T	bash
root	1288	1044	0.0	0.0	Z	[defunct]

7. WHY A PROCESS CAN BE “ALIVE” BUT UNHEALTHY

- A process in sleeping (S) or uninterruptible sleep (D) might be stuck waiting for I/O.
- A process in stopped (T) state is alive but not running.
- A zombie (Z) process has finished but is still listed until the parent collects its exit status.
- A process can be using 0% CPU but still be unhealthy (e.g. waiting for a dead network service).

8. COMMON PROBLEMS

- High number of processes in D state – storage or network issue.
- Many zombie processes – parent not collecting exit status.
- Processes stuck in S state – waiting for a resource or event.
- Process in T state – accidentally stopped (SIGSTOP).

9. TROUBLESHOOTING TIPS

- Check what the process is waiting for (I/O, network, lock).
- Look at system logs and application logs.
- Check disk, network and dependencies.
- Use `strace` to see what the process is waiting on.

10. KEY TAKEAWAY



A process being visible and “alive” does not mean it is healthy. Always check the process state and understand what it is waiting for or doing.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CPU Scheduling & Load



How runnable processes compete for CPU, scheduler basics, context switching, CPU-bound workloads and why load average is not simply CPU percentage.

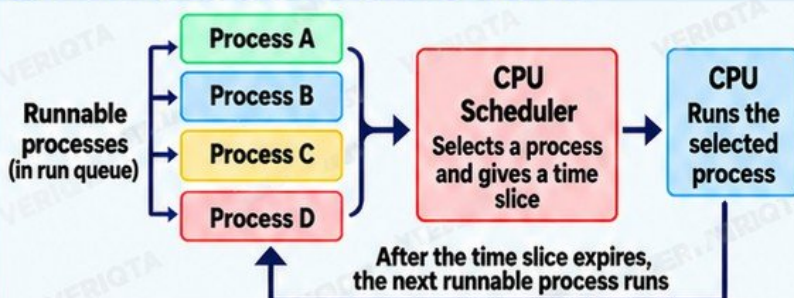
1. WHAT IS CPU SCHEDULING?

- The CPU scheduler decides which runnable process gets CPU time and for how long.
- Multiple processes compete for CPU.
- Each process gets a small time slice (time quantum).
- The scheduler switches between processes very quickly (context switching).
- This gives the illusion that multiple processes run at the same time.

2. KEY CONCEPTS

- **Runnable:** process is ready to run and waiting for CPU.
- **Time slice:** small period of CPU time given to a process.
- **Context switch:** saving the current process state and loading another process state.
- **CPU-bound:** process uses CPU most of the time (e.g. data processing, compression).
- **I/O-bound:** process waits for I/O most of the time (e.g. read from disk, network requests).

3. HOW CPU SCHEDULING WORKS



4. CONTEXT SWITCHING

- The kernel saves the state (registers, memory pointers, etc.) of the current process.
- It loads the state of the next process.
- This takes a small amount of time (context switch overhead).
- Frequent context switching can reduce performance, especially in CPU-bound workloads.

5. CPU-BOUND vs I/O-BOUND

CPU-BOUND

- Uses CPU most of the time.
- Example: video encoding, data processing, compression.
- Creates high CPU usage.
- Benefits from multiple CPU cores.

I/O-BOUND

- Spends a lot of time waiting for I/O (disk, network, etc.).
- Example: web servers, databases, file operations.
- Lower CPU usage.
- Performance depends on I/O subsystem, not CPU.

6. UNDERSTANDING LOAD AVERAGE

- Load average shows the average number of runnable processes over a time period (1, 5, 15 minutes).
- It includes processes that are:
 - running (R state)
 - waiting for CPU (runnable)
- It does NOT show CPU percentage.
- A high load average means more processes want CPU than available CPU cores.
- It can be high even if CPU usage is low (e.g. many processes waiting for I/O but in runnable state).

7. EXAMPLE OUTPUT (uptime and top)

```
$ uptime
```

```
10:24:01 up 2 days, 3:15, 3 users, load average: 2.45, 1.98, 1.56
```

Load average for 1, 5 and 15 minutes

```
$ top - 05:24:10
```

```
Tasks: 232 total, 4 running, 228 sleeping, 0 stopped, 0 zombie
%Cpu(s): 72.3 us, 5.1 sy, 0.0 ni, 22.1 id, 0.3 wa, 0.0 hi, 0.2 si, 0.0 st
```

8. KEY TAKEAWAYS

- The CPU scheduler gives each runnable process a time slice.
- Context switching allows multiple processes to share the CPU.
- Load average measures demand for CPU, not actual CPU usage.
- Understand process type (CPU-bound vs I/O-bound) to troubleshoot performance.

9. TRY THIS

- 1 Run **uptime** and note the load average.
- 2 Run **top** and check running processes and CPU usage.
- 3 Start a CPU-bound task (e.g. `yes > /dev/null`) and observe the load average.
- 4 Start an I/O-heavy task (e.g. `dd if=/dev/zero of=test bs=1M count=1000`) and observe the difference.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Memory: Virtual vs Physical



RAM, virtual address spaces, pages, mappings and why every process appears to have its own memory space.

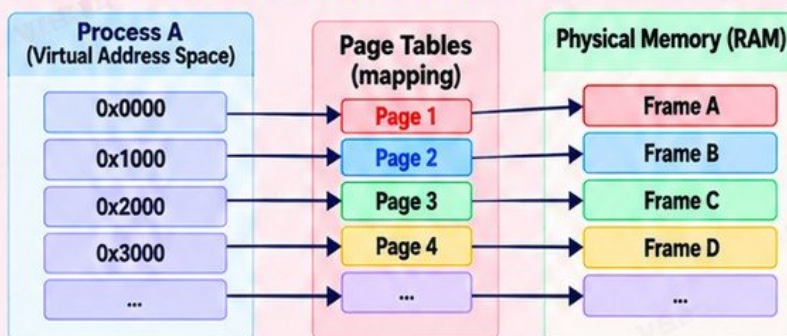
1. KEY CONCEPTS

- **Physical memory (RAM)** is the actual hardware memory in the server.
- **Virtual memory** is a large address space presented to each process by the operating system.
- Each process gets its own virtual address space.
- The kernel maps virtual addresses to physical memory using pages.
- This allows isolation, security and efficient memory usage.

2. VIRTUAL VS PHYSICAL MEMORY

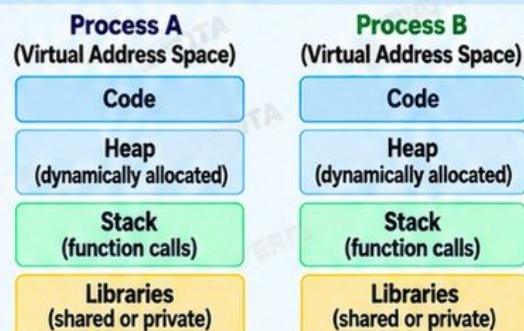
Virtual Memory	Physical Memory (RAM)
Provided by the OS (kernel).	Actual hardware memory on the server.
Each process has its own virtual address space.	Shared by all processes.
Much larger than RAM (e.g. 128GB virtual on a machine with 16GB RAM).	Limited size (e.g. 16GB, 32GB, 64GB).
Mapped to physical memory using pages.	Stores data for processes, kernel and cache.
Provides isolation and security.	Faster to access than disk swap.

3. HOW IT WORKS (AT A HIGH LEVEL)



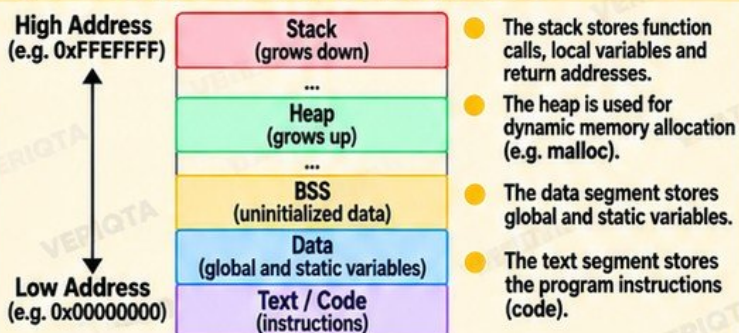
- 1 The process uses a virtual address (e.g. 0x1000).
- 2 The CPU and kernel use page tables to translate it to a physical address.
- 3 The data is read from or written to the physical memory frame.

4. EVERY PROCESS HAS ITS OWN MEMORY SPACE



- Each process thinks it has its own memory space.
- The same virtual addresses in different processes map to different physical memory.
- This provides isolation and prevents one process from directly accessing another process's memory.

5. MEMORY LAYOUT OF A PROCESS



6. PAGES AND MAPPINGS

- Memory is divided into fixed-size blocks called pages (usually 4KB).
- Page tables map virtual pages to physical frames.
- Only the pages that are currently needed are loaded into RAM.
- Pages not in RAM can be stored on disk (swap).
- This allows processes to use more memory than physical RAM.

7. KEY TAKEAWAYS

- Virtual memory gives each process its own address space.
- The kernel maps virtual addresses to physical memory using pages.
- This provides isolation, security and efficient memory usage.
- A process can use more memory than available RAM because not all pages are in RAM.

8. REAL DEVOPS CONTEXT

- Containers and processes rely on the same virtual memory mechanism.
- Memory isolation helps prevent one application from crashing another.
- Understanding virtual vs physical memory is important for troubleshooting memory issues, performance and container limits.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Linux Memory in Practice

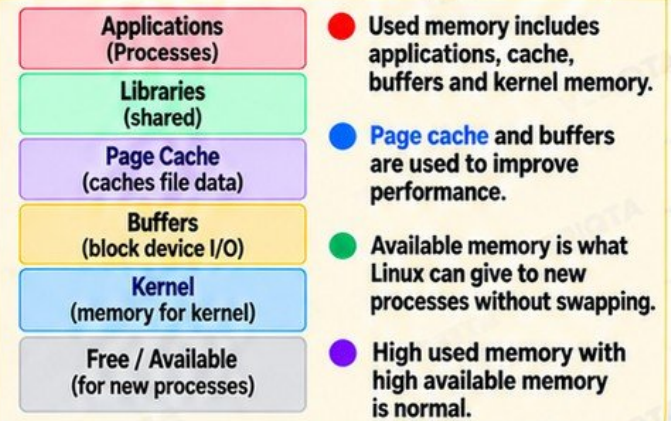


Used, available, buffers, page cache, RSS and swap. Teach learners how to interpret free, /proc/meminfo and ps without panicking over "used" RAM.

1. KEY MEMORY TERMS

Total	Total physical RAM in the system.
Used	Memory currently used (includes cache and buffers).
Free	Memory not used at all.
Available	Memory available for new applications (without swapping).
Buffers	Used for block device I/O operations.
Cache (Page Cache)	Used to cache file data for faster access.
RSS	Actual physical memory used by a process.
Swap	Disk space used when RAM is insufficient.

2. LINUX MEMORY LAYOUT (HIGH LEVEL)



3. UNDERSTANDING free OUTPUT

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	15Gi	8.1Gi	1.2Gi	0.8Gi	5.7Gi	6.2Gi
Swap:	4.0Gi	0B	4.0Gi			

- **used:** includes applications, page cache, buffers and kernel memory.
- **free:** memory not used at all (usually small).
- **buff/cache:** memory used for caching files and buffers (reclaimable).
- **available:** memory available for new applications without swapping.

4. UNDERSTANDING /proc/meminfo

```
$ grep -E 'MemTotal|MemFree|MemAvailable|Buffers|Cached|Swap' \
/proc/meminfo
```

```
MemTotal:        16384256 kB
MemFree:         1256780 kB
MemAvailable:    6523412 kB
Buffers:         312456 kB
Cached:          5843210 kB
SwapTotal:      4194304 kB
SwapFree:       4194304 kB
```

- **MemAvailable** shows how much memory can be used by new processes.
- **Cached** includes page cache used for file data.
- **Buffers** are used for block device I/O.
- **SwapTotal** and **SwapFree** show swap usage.

5. UNDERSTANDING ps OUTPUT (PROCESS MEMORY)

```
$ ps aux --sort=-%mem | head
```

USER	PID	%CPU	%MEM	RSS	VSZ	COMMAND
root	1288	0.2	5.1	812M	1.2G	java
veriqta	2341	1.0	3.8	612M	1.0G	nginx
root	4321	0.1	2.2	356M	980M	postgres
root	5678	0.0	1.4	220M	450M	python
veriqta	9012	0.3	0.9	140M	320M	node

- **RSS:** actual physical memory used by the process (non-swapped).
- **VSZ:** total virtual memory size (includes memory not yet loaded).
- **%MEM:** percentage of total RAM used by the process.
- High VSZ does not always mean high actual memory usage.

6. COMMON SCENARIOS (AND WHAT THEY MEAN)

Scenario	Explanation
High used, high available	Normal. Memory is being used for cache which can be released when needed.
Low free memory	Normal. Linux uses free memory for caching.
High swap usage	System is under memory pressure.
Increasing used and decreasing available	May indicate a memory leak or high workload.
High RSS for a process	Process is using a lot of physical memory.
High VSZ but low RSS	Process has a large virtual address space but not much is loaded into RAM.

7. KEY TAKEAWAYS

- Do not panic when used memory is high.
- Look at available memory, not just free.
- Understand the difference between cache, buffers and real application memory (RSS).
- Use multiple tools (free, /proc/meminfo, ps).
- High memory usage can be normal if the system has high available memory.

8. TROUBLESHOOTING TIPS

- Check for memory leaks if available memory keeps decreasing.
- Identify memory-heavy processes using **ps** or **top**.
- Monitor swap usage and investigate if it keeps growing.
- Check application logs for unusual memory behaviour.
- Consider whether high memory usage is expected for the workload.

9. TRY THIS

- Run **free -h** and explain each column.
- Check **/proc/meminfo** and find **MemAvailable**.
- Run **ps aux --sort=-%mem | head** and identify the top memory users.
- Compare **VSZ** and **RSS** for a large process.
- Observe how available memory changes after reading a large file (uses page cache).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Memory Pressure & the OOM Killer

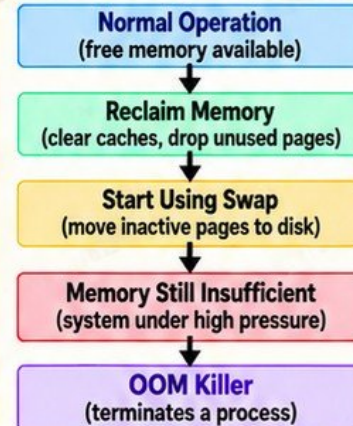


What happens when memory becomes scarce, reclaim, swapping, out-of-memory conditions, OOM victim selection and evidence engineers should inspect.

1. WHAT IS MEMORY PRESSURE?

- Memory pressure happens when the system does not have enough free memory to meet demand.
- The kernel starts reclaiming memory from caches, inactive pages and unused memory.
- If reclaim is not enough, the system starts using swap.
- When memory is still insufficient, the kernel can invoke the OOM killer to terminate a process.
- Memory pressure can cause slow applications, high latency and timeouts even before the OOM killer acts.

2. MEMORY PRESSURE FLOW



WHAT THE SYSTEM DOES

- 1 Reclaims page cache and unused memory.
- 2 Swaps inactive pages to disk.
- 3 If memory is still low, triggers the OOM killer.
- 4 Selects a process to terminate.
- 5 Frees memory and continues running.

3. THE OOM KILLER

- The OOM (Out Of Memory) killer is a kernel mechanism that terminates a process when memory is critically low.
- It selects a victim based on factors such as memory usage, OOM score and the process's importance.
- The goal is to free enough memory for the system to continue running, not to fix the underlying problem.
- The killed process is usually the one using the most memory or has a high OOM score.

4. OOM VICTIM SELECTION FACTORS

- Amount of memory used by the process.
- OOM score (in `/proc/[pid]/oom_score`).
- OOM score adjustment (`oom_score_adj`).
- Whether the process is critical (system processes are protected).
- Overall system memory pressure.
- The process that is easiest to kill to free enough memory.

5. MEMORY PRESSURE STATES



6. CHECKING MEMORY PRESSURE

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	15Gi	14Gi	500Mi	1.2Gi	3.5Gi	1.1Gi
Swap:	4.0Gi	2.1Gi	1.9Gi			

- Check total, used, free, buff/cache and available memory.
- If available is very low, the system is under memory pressure.

```
$ cat /proc/meminfo | head
```

```
MemTotal:      16384256 kB
MemFree:       512008 kB
MemAvailable:  1123456 kB
Buffers:       320456 kB
Cached:       2845320 kB
SwapTotal:    4194304 kB
SwapFree:     1999808 kB
```

- Shows detailed memory information.
- MemAvailable is a good indicator of how much memory can be used by new processes.

```
$ cat /proc/<pid>/oom_score
862
```

- Shows the OOM score for a process (higher value means more likely to be killed).

8. COMMON SIGNS OF MEMORY PRESSURE

- System becomes slow or unresponsive.
- High swap usage (`vmstat`, `free`, `top`).
- Applications timing out or crashing.
- OOM killer messages in system logs.
- High memory usage processes (`top`, `ps`, `htop`).

9. TROUBLESHOOTING TIPS

- Identify top memory using processes (`top`, `ps`).
- Check if memory usage is expected (e.g. caching, large workloads).
- Look at swap usage and I/O wait.
- Check application and system logs.
- Review OOM killer logs in `dmesg`.
- Consider increasing memory, reducing workload or fixing memory leaks.

7. OOM KILLER EVIDENCE (LOGS)

```
$ dmesg | grep -i out of memory
```

```
Out of memory: Killed process 1234 (python)
total-vm:2097152kB, anon-rss:1024000kB,
file-rss:0kB, shmem-rss:0kB, UID:1000 pgtables:2048kB
oom_score_adj:0
```

- Look for "Out of memory: Killed process" in `dmesg` or `/var/log/kern.log`.
- The log shows the killed process, PID, memory usage and OOM score.
- This helps you understand what was killed and why.

10. KEY TAKEAWAYS

- Linux reclaims memory and uses swap before invoking the OOM killer.
- The OOM killer terminates a process to keep the system running.
- High memory usage does not always mean a problem (caches are normal).
- Always check logs and understand the cause, not just the symptom.
- Prevent memory issues by right-sizing applications and monitoring memory usage.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

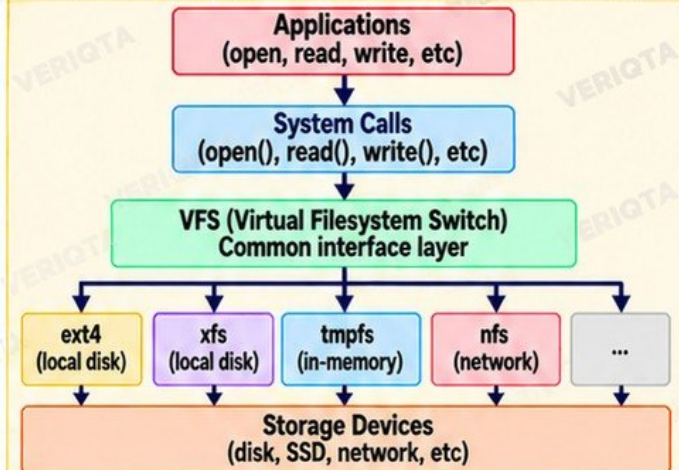
The Linux Filesystem Model

VFS concept, files, directories, inodes, metadata and directory entries. Explain why a filename and the underlying file are not the same thing.

1. WHAT IS THE LINUX FILESYSTEM MODEL?

- Linux presents a single unified directory tree that starts at / (root).
- The Virtual Filesystem Switch (VFS) is a layer in the kernel that provides a common interface for different filesystems (ext4, xfs, btrfs, tmpfs, etc).
- Files and directories are not the data itself. They are objects that reference inodes.
- The model separates the filename (directory entry) from the actual file (inode and data).
- This allows flexibility, multiple filesystems and consistent behaviour across storage types.

2. LINUX FILESYSTEM ARCHITECTURE (VFS)



3. KEY COMPONENTS

File

- A collection of data.
- Represented by an inode in the filesystem.
- Can be a regular file, block device, character device, socket, etc.

Directory

- A special file that contains directory entries (name → inode).
- Maps human readable names to inode numbers.
- Just like a file, it also has its own inode.

Inode

- Stores metadata about a file.
- Contains file type, permissions, owner, size, timestamps and pointers to data blocks on disk.
- The actual file content is identified by its inode number.

Directory Entry

- A record that maps a filename to an inode number.
- A filename is just a name in a directory entry.
- Multiple directory entries (can be in different directories) can point to the same inode (hard links).

4. FILENAME vs INODE (NOT THE SAME THING)

Directory (e.g. /etc)

nginx → inode 1234
hosts → inode 5678
app.conf → inode 9012

Inode 1234 (metadata)

- File type: regular file
- Permissions: -rw-r--r--
- Owner: root
- Size: 4KB
- Timestamps: atime, mtime, ctime
- Pointers to data blocks

Data Blocks (actual content)

Block 1
Block 2
Block 3
...

- A filename is just a name used to find an inode.
- The inode stores the file's metadata and location of data.
- The actual file content is stored in data blocks.
- You can have multiple filenames (hard links) pointing to the same inode.
- Deleting a filename removes the directory entry, but the file still exists if another filename points to the same inode (or if a process has it open).

5. REAL EXAMPLE (INODE AND DIRECTORY ENTRIES)

```
$ ls -li
1234 -rw-r--r-- 1 root root 4096 Oct 1 10:00 file1.txt
1234 -rw-r--r-- 1 root root 4096 Oct 1 10:00 file2.txt
```

- file1.txt and file2.txt are different names (directory entries) but point to the same inode (1234). They are hard links to the same file.

6. USEFUL COMMANDS

Command	Purpose
ls -li	Show inode numbers and file details.
stat <file>	Show detailed inode information.
df -T	Show mounted filesystems and types.
findmnt	Show the filesystem mount tree.
cat /proc/filesystems	List supported filesystems in the kernel.
mount column -t	Show currently mounted filesystems.
df -h	Show disk usage per filesystem.

7. HARD LINKS vs SYMBOLIC LINKS

Hard Link

- Another directory entry pointing to the same inode.
- Both names share the same file data.
- If one name is deleted, the file still exists if the other name remains.

Symbolic Link (Symlink)

- A separate file that contains a path to another file.
- Has its own inode.
- If the target file is deleted, the symlink becomes a broken link.

8. KEY TAKEAWAYS

- Linux uses a single directory tree with a VFS layer.
- A file is identified by its inode, not by its filename.
- A directory contains entries that map names to inode numbers.
- A filename and the underlying file are not the same thing.
- Multiple names can point to the same inode.

9. COMMON SCENARIOS

- Deleting a file but it still exists (open by a process).
- Multiple filenames for the same file (hard links).
- Broken symlinks (target removed).
- Understanding disk usage and inode usage during troubleshooting.
- Different filesystems (ext4, xfs, tmpfs, nfs) all work through the same VFS interface.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

File Descriptors



stdin, stdout, stderr, sockets, pipes and open files. Connect file descriptors to applications, logs, networking and “too many open files” incidents.

1. WHAT ARE FILE DESCRIPTORS?

- A file descriptor (FD) is a small integer used by a process to reference an open file, socket, pipe or other I/O resource.
- When a process opens a file or creates a connection, the kernel returns a file descriptor.
- File descriptors are per process. Each process has its own table of open file descriptors.
- FDs are not just for files. They can represent files, directories, sockets, pipes, terminals and devices.
- When a process no longer needs the resource, it closes the file descriptor.

2. STANDARD FILE DESCRIPTORS

FD	Name	Purpose	Common Use
0	stdin	Standard input	Read input from keyboard, file or pipe.
1	stdout	Standard output	Normal output (e.g. program results, logs).
2	stderr	Standard error	Error messages and warnings.

```
$ command > output.txt 2> error.txt
```

stdout (1)
goes to output.txt

stderr (2)
goes to error.txt

3. TYPES OF FILE DESCRIPTORS

Regular Files



- Used to read and write files on disk.
- Example: open(), read(), write().

Directories



- Also represented by file descriptors.
- Used for reading directory entries (e.g. readdir()).

Sockets



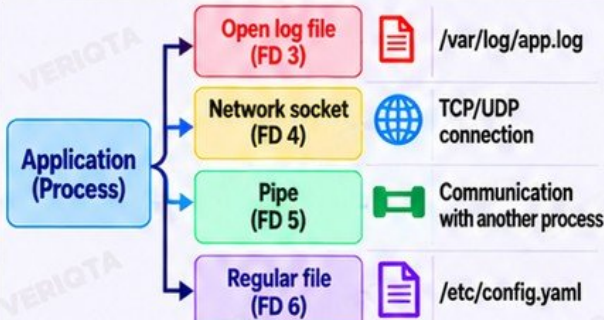
- Network connections (TCP/UDP).
- Used for client and server communication.

Pipes



- Connect the output of one process to the input of another.
- Used for inter-process communication (IPC).

4. FILE DESCRIPTORS IN ACTION



5. VIEWING OPEN FILE DESCRIPTORS

```
$ ls -l /proc/$$/fd
0 -> /dev/pts/0
1 -> /dev/pts/0
2 -> /dev/pts/0
3 -> /var/log/app.log
4 -> socket:[4026532583]
5 -> pipe:[4026532584]
```

- Shows the open file descriptors for the current process.
- Each number is a file descriptor.
- You can see files, sockets, pipes and other resources.
- This is useful for debugging resource usage.

6. REAL EXAMPLES

- Application Logs → File descriptor to a log file (e.g. /var/log/app.log).
- Networking → File descriptor to a socket (client or server connection).
- Pipelines → File descriptors for pipes (e.g. command1 | command2).
- Web Servers → Many file descriptors for client connections (sockets).
- Databases → File descriptors for data files, logs and network connections.

7. “TOO MANY OPEN FILES” INCIDENTS

- Each process has a limit on the number of open file descriptors (ulimit -n).
- When the limit is reached, the process cannot open new files, sockets or connections.
- This can cause application failures, failed connections and service outages.
- Common causes include file descriptor leaks, high traffic, many open network connections or not closing files properly.

```
$ cat large_file.txt
cat: large_file.txt:
Too many open files
```

The process has reached its file descriptor limit and cannot open new files.

8. KEY TAKEAWAYS

- A file descriptor is a small integer that references an open resource.
- stdin (0), stdout (1) and stderr (2) are always available.
- File descriptors can represent files, directories, sockets, pipes and devices.
- You can inspect open file descriptors using /proc/<pid>/fd and tools like lsof.
- “Too many open files” happens when the process reaches its file descriptor limit.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



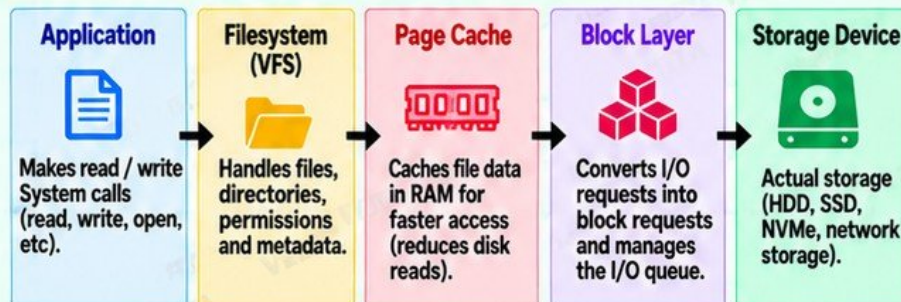
@veriqta

Disk I/O & Storage Internals



Application → filesystem → page cache → block layer → storage device.
Introduce latency, throughput, I/O wait and diagnosing slow storage.

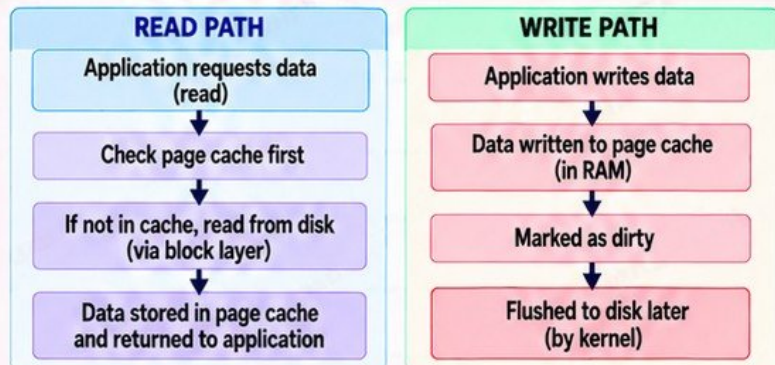
1. THE DISK I/O FLOW (HIGH LEVEL)



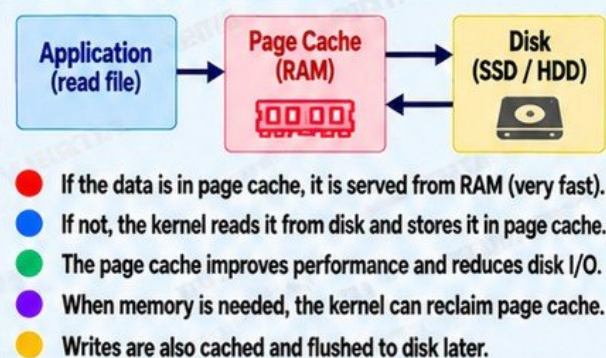
2. KEY CONCEPTS

- Latency:** time taken to complete a single I/O operation (milliseconds).
- Throughput:** amount of data transferred per second (MB/s).
- I/O wait:** time the CPU waits for disk I/O to complete.
- Page cache:** keeps recently read and written data in RAM.
- Block layer:** manages and queues I/O requests.
- Storage device:** the actual hardware (HDD, SSD, NVMe or network storage).

3. READ vs WRITE PATH



4. PAGE CACHE IN ACTION



5. COMMON TOOLS TO CHECK DISK I/O

1. iostat (overall disk I/O)

```
$ iostat -x 1
```

Device	r/s	w/s	rMB/s	wMB/s	await
sda	12.3	45.6	5.2	12.4	4.8
nvme0n1	100.2	80.1	220.5	180.3	1.2

2. vmstat (I/O wait and system view)

```
$ vmstat 1
```

procs		memory		swap	io	system		cpu	
r	b	si	so	bi	bo	us	sy	id	wa
2	0	0	0	1024	2048	5	3	89	3

3. iotop (per process I/O)

```
$ sudo iotop
```

PID	USER	DISK READ	DISK WRITE	COMMAND
4321	veriqta	12.5 M/s	1.2 M/s	java
5678	root	0.0 B/s	8.4 M/s	postgres

4. lsblk (storage devices)

```
$ lsblk
```

NAME	SIZE	TYPE	MOUNTPOINT
sda	100G	disk	
└─sda1	100G	part	/
nvme0n1	1T	disk	
└─nvme0n1p1	1T	part	/data

8. COMMON SYMPTOMS

- System is slow but CPU usage is low.
- High I/O wait in top or vmstat.
- High await/latency in iostat.
- Applications timing out.
- Databases or web servers responding slowly.
- High disk queue and large read/write rates.

9. KEY TAKEAWAYS

- Disk I/O goes through application → filesystem → page cache → block layer → storage device.
- Page cache greatly improves performance.
- Latency and throughput are different.
- High I/O wait is a sign of storage bottleneck.
- Use the right tools to identify the cause.
- Always check the underlying storage type (HDD, SSD, NVMe or network storage).

6. UNDERSTANDING I/O WAIT

- I/O wait (wa) is the percentage of time the CPU is waiting for disk I/O to complete.
- High I/O wait usually means slow storage or heavy disk load.
- A system can have low CPU usage but still be slow due to high I/O wait.
- Common causes: slow disks, high disk queue, large reads/writes, or too many I/O requests.

7. DIAGNOSING SLOW STORAGE

- Check I/O wait: vmstat 1 or top
- Check disk latency and throughput: iostat -x 1
- Find processes causing I/O: iotop
- Check disk queue length: iostat or sar -d
- Check filesystem usage and type: df -h, mount
- Look at storage type: HDD (slower) vs SSD/NVMe (faster)
- Check application logs for slow I/O or timeouts

10. REAL-WORLD EXAMPLE

- A web application becomes slow during peak hours.
- CPU usage is low but I/O wait is high.
 - iostat shows high disk latency.
 - iotop reveals the database process doing heavy disk writes.
 - The root cause is a slow disk or high disk load.
 - Solution: optimize queries, improve storage (SSD/NVMe) or reduce unnecessary writes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

/proc: Looking Inside Linux

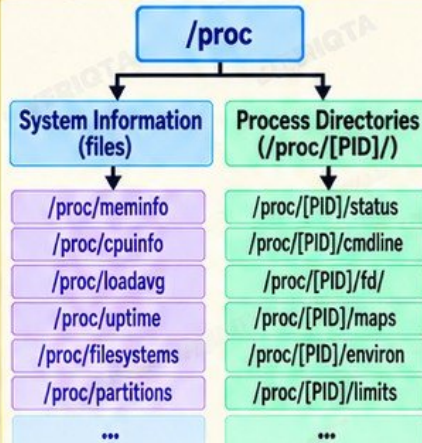


/proc, process directories, status, cmdline, fd, maps, meminfo, cpuinfo and how engineers inspect kernel/process state without special monitoring software.

1. WHAT IS /proc?

- /proc is a virtual filesystem that provides a view into kernel and process information.
- It does not contain real files on disk. The contents are generated by the kernel in real time.
- It allows engineers to inspect system and process state without installing any additional monitoring software.
- It is widely used for troubleshooting, performance analysis and automation.

2. /proc STRUCTURE OVERVIEW



3. KEY /proc FILES (SYSTEM)

File	Purpose
/proc/meminfo	System memory information.
/proc/cpuinfo	CPU details.
/proc/loadavg	System load average.
/proc/uptime	System uptime.
/proc/filesystems	Supported filesystems.
/proc/partitions	Disk partitions.
/proc/version	Kernel version.
/proc/sys/	Kernel parameters and tunables.
/proc/modules	Loaded kernel modules.
/proc/mounts	Mounted filesystems.

4. PROCESS DIRECTORIES

- Each running process has a directory named with its PID (e.g. /proc/1234).
- These directories contain detailed information about that process.
- When the process exits, the directory disappears.

```
$ ls /proc | head
1      2      3      4      5
6      7      8      9      10
123    456    789    1024   2048
3123   4321   5678   9012   ...
```

5. IMPORTANT PROCESS FILES

File / Directory	Purpose
status	Process status, memory, state and more.
cmdline	Command line used to start the process.
fd/	Open file descriptors (files, sockets, pipes).
maps	Memory mappings (libraries, heap, stack).
environ	Environment variables.
limits	Resource limits (e.g. open files, memory).
cwd	Current working directory (symlink).
exe	Path to the executable (symlink).

6. PRACTICAL EXAMPLES

View process status

```
$ cat /proc/1234/status
Name:      nginx
State:     S (sleeping)
Pid:       1234
PPid:      1
VmRSS:     25600 kB
```

View process command line

```
$ cat /proc/1234/cmdline
/usr/sbin/nginx -g\
daemon off;
```

View open file descriptors

```
$ ls -l /proc/1234/fd
0 -> /dev/null
1 -> /var/log/nginx/access.log
2 -> /var/log/nginx/error.log
3 -> socket:[402653248]
4 -> anon_inode:[eventpoll]
```

View memory mappings

```
$ cat /proc/1234/maps | head
55e1c3c00000-55e1c3c21000 r--p ...
7f2a4d400000-7f2a4d600000 r-xp ...
```

View environment variables

```
$ cat /proc/1234/environ | tr '\0' '\n'
PATH=/usr/local/sbin:/usr/local/bin
USER=nginx
...
```

View current working directory and exe

```
$ ls -l /proc/1234/{cwd,exe}
cwd -> /etc/nginx
exe -> /usr/sbin/nginx
```

7. INSPECTING SYSTEM STATE

```
# Memory information
$ cat /proc/meminfo | head
# CPU information
$ cat /proc/cpuinfo | head
# Load average
$ cat /proc/loadavg
# Mounted filesystems
$ cat /proc/mounts | head
```

8. REAL-WORLD USE CASES

- Investigate high memory usage for a process (/proc/[PID]/status, maps).
- Find what files or sockets a process has open (/proc/[PID]/fd).
- Check the exact command used to start a process (/proc/[PID]/cmdline).
- Understand system load and CPU details (/proc/loadavg, /proc/cpuinfo).
- Check mounted filesystems and storage information (/proc/mounts, /proc/partitions).

9. KEY TAKEAWAYS

- /proc provides real-time information about the kernel and running processes.
- Process directories contain everything you need to understand a process.
- You can inspect memory, open files, network connections and more without extra tools.
- It is essential for troubleshooting and performance analysis.
- The data is generated by the kernel and disappears when the process exits.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

/sys, Devices & the Kernel

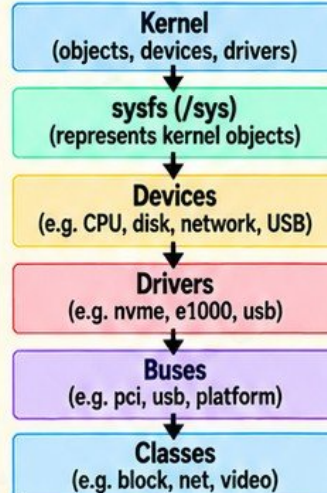


What sysfs exposes, device representation, kernel objects and the difference between /proc, /sys and /dev.

1. WHAT IS /sys?

- /sys is a virtual filesystem (sysfs) that exposes kernel objects, devices, drivers and their attributes.
- It provides a structured view of the kernel's internal device and driver model.
- Each kernel object (device, driver, bus, class, etc.) is represented as a directory with attributes (files).
- You can read or sometimes write to these attributes to view or configure kernel settings in real time.
- It is widely used for hardware information, device management and system configuration.

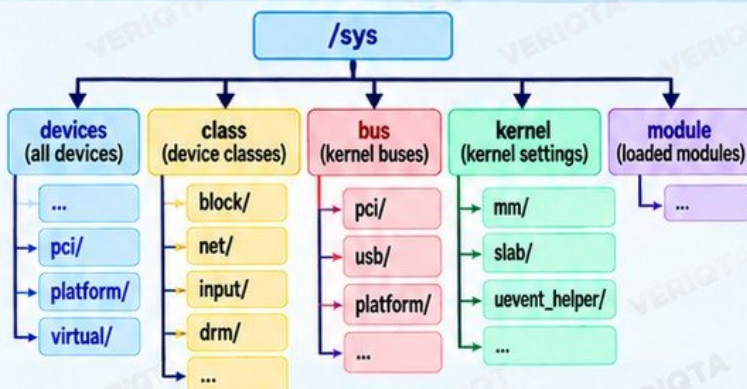
2. KERNEL DEVICE MODEL



3. /proc vs /sys vs /dev (KEY DIFFERENCES)

Path	What it is	What it shows	Example
/proc	Virtual filesystem	Process and system runtime information (generated by kernel)	/proc/cpuinfo /proc/meminfo /proc/[PID]/ /proc/loadavg
/sys	Virtual filesystem (sysfs)	Kernel objects, devices, drivers and attributes (structured view of kernel)	/sys/devices/ /sys/class/ /sys/bus/ /sys/kernel/
/dev	Device filesystem (devtmpfs)	Device files that provide interfaces to hardware and virtual devices	/dev/sda /dev/tty /dev/null /dev/urandom

4. /sys DIRECTORY STRUCTURE



5. REAL EXAMPLES

1. List devices in /sys

```
$ ls /sys/devices | head
breakpoint
cpu
pci
platform
virtual
```

2. CPU information

```
$ ls /sys/devices/system/cpu/
cpu0
cpu1
cpu2
cpu3
...
```

3. Block devices (disks)

```
$ ls /sys/class/block/
sda
sdb
nvme0n1
loop0
```

4. Network devices

```
$ ls /sys/class/net/
eth0
lo
...
```

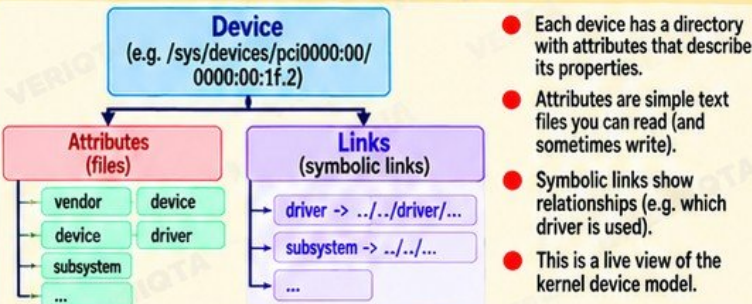
5. Read device attribute

```
$ cat /sys/class/block/sda/queue/rotational
1 # 1 = HDD, 0 = SSD
```

6. View loaded kernel modules

```
$ ls /sys/module | head
acpi
aesni_intel
autofs4
...
```

6. DEVICE REPRESENTATION IN /sys



- Each device has a directory with attributes that describe its properties.
- Attributes are simple text files you can read (and sometimes write).
- Symbolic links show relationships (e.g. which driver is used).
- This is a live view of the kernel device model.

7. COMMON USE CASES

- Identify and inspect hardware devices.
- Check which driver is in use.
- View and modify kernel parameters.
- Troubleshoot device and driver issues.
- Monitor device state and power management.
- Used by system tools (udev, lspci, lsblk, ip, ethtool) internally.

8. KEY TAKEAWAYS

- /sys exposes kernel objects, devices and drivers.
- It provides a structured, real-time view of the kernel device model.
- /proc is for process and system runtime information.
- /dev provides device files for interacting with hardware and virtual devices.
- Understanding these filesystems helps with debugging, performance analysis and system administration.

9. USEFUL COMMANDS

Command	Purpose
ls /sys/devices/	List all devices.
ls /sys/class/	List device classes.
ls /sys/bus/	List kernel buses.
cat /sys/class/net/eth0/operstate	Check network state.
cat /sys/block/sda/queue/rotational	Check if disk is SSD or HDD.
lsmod	View loaded kernel modules.
udevadm info /dev/sda	Device information via udev.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



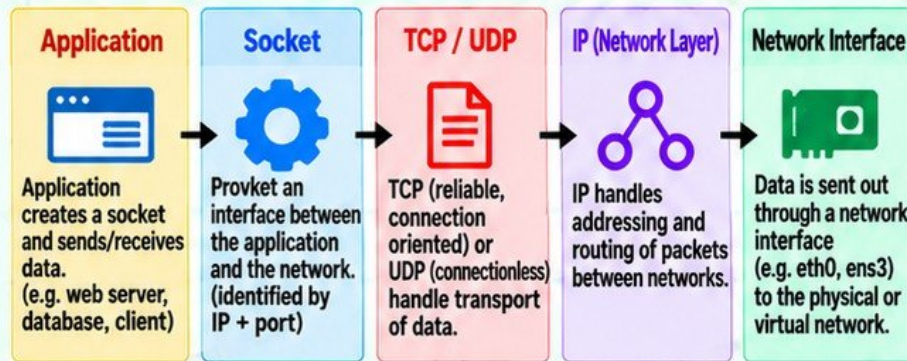
@veriqta

Linux Networking Internals

Application → socket → TCP/UDP → IP → network interface.

Connect ports, listening sockets, connections and interfaces to real application troubleshooting.

1. NETWORKING DATA FLOW (HIGH LEVEL)



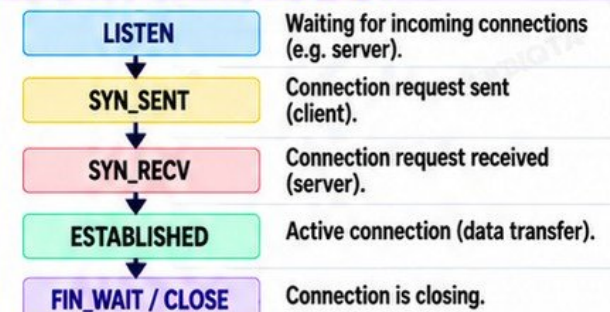
2. KEY NETWORKING CONCEPTS

- **IP address:** unique address for a device on a network.
- **Port:** identifies an application or service on a host.
- **Socket:** combination of IP address and port (e.g. 0.0.0.0:80).
- **TCP:** reliable, connection oriented, used for web, databases, file transfer, etc.
- **UDP:** connectionless, faster, used for DNS, streaming, VoIP, etc.
- **Listening socket:** waiting for incoming connections.
- **Established connection:** active communication between client and server.
- **Network interface:** hardware or virtual device (e.g. eth0, ens3, lo).

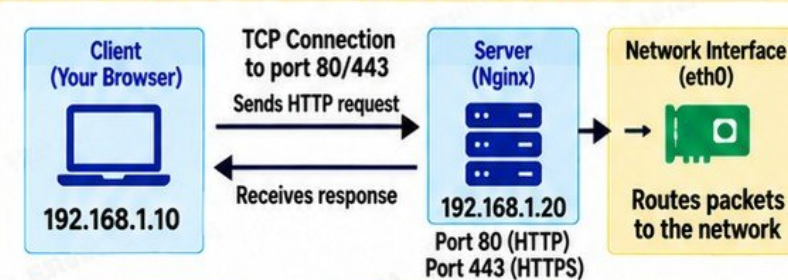
3. TCP vs UDP

TCP	UDP
● Connection oriented	● Connectionless
● Reliable delivery	● No guarantee of delivery
● Maintains connection state	● No connection state
● Handles packet loss and ordering	● Faster and lower overhead
● Used for HTTP, HTTPS, SSH, Databases, etc.	● Used for DNS, streaming, online gaming, VoIP, etc.

4. SOCKET STATES (TCP)



5. REAL EXAMPLE: WEB SERVER CONNECTION



6. USEFUL COMMANDS

Command	Purpose
<code>ss -tulnp</code>	Show listening sockets and ports.
<code>ss -tnp</code>	Show active TCP connections.
<code>ss -unp</code>	Show active UDP connections.
<code>netstat -tulnp</code>	Alternative to ss (older systems).
<code>lsof -i</code>	Show processes using network ports.
<code>ip addr</code>	Show network interfaces and IP addresses.
<code>ip route</code>	Show routing table.
<code>ping <host></code>	Test network connectivity.
<code>traceroute <host></code>	Trace the network path to a host.
<code>curl -v <url></code>	Test HTTP/HTTPS connection.
<code>nc -zv <host> <port></code>	Check if a port is open.

7. EXAMPLE COMMAND OUTPUTS

1. Listening sockets	2. Active connections
<pre>\$ ss -tulnp Netid State Local Address:Port tcp LISTEN 0.0.0.0:80 tcp LISTEN 0.0.0.0:22 udp UNCONN 0.0.0.0:53</pre>	<pre>\$ ss -tnp State Local Address:Port Peer Address:Port ESTAB 192.168.1.20:80 192.168.1.10:52344 ESTAB 192.168.1.20:22 192.168.1.15:60123</pre>
3. Network interfaces	4. Routing table
<pre>\$ ip addr show 2: eth0: <BROADCAST,MULTICAST,UP> inet 192.168.1.20/24 inet6 fe80::a00:27ff:fe4e:66a1/64</pre>	<pre>\$ ip route default via 192.168.1.1 dev eth0 192.168.1.0/24 dev eth0 proto kernel 10.0.0.0/24 dev eth1 proto kernel</pre>

8. COMMON TROUBLESHOOTING SCENARIOS

- Port not listening: application not running or misconfigured.
- Connection refused: no service listening on the port.
- Connection timeout: network issue, firewall or unreachable host.
- High number of connections: check for traffic spike or connection leak.
- Slow network: check latency, packet loss and interface errors.
- Cannot access service: check firewall rules, security groups and routing.

9. KEY TAKEAWAYS

- An application uses a socket to send and receive data.
- TCP provides reliable, connection oriented communication.
- UDP is connectionless and faster.
- Sockets are identified by IP address and port.
- Use tools like ss, lsof, ip and curl to inspect and troubleshoot.
- Network interfaces connect your system to the network.
- Understanding this flow helps you quickly diagnose real application networking issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Signals & Process Control

SIGTERM, SIGKILL, SIGHUP, SIGINT, graceful termination and why kill -9 should not be the automatic response to a troublesome process.

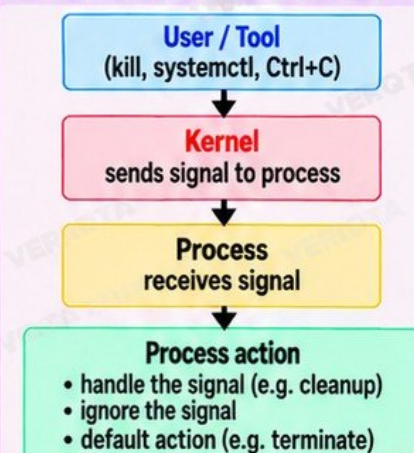
1. WHAT ARE SIGNALS?

- Signals are software interrupts sent to a process to notify it of an event.
- They allow you to control processes (e.g. stop, reload, terminate).
- A process can choose to handle a signal (graceful action) or ignore it.
- Signals are widely used by the kernel, shell, systemd and applications.
- They are different from normal data — they do not carry a long message, just a signal number.

2. COMMON SIGNALS

Signal	Number	Purpose
SIGHUP	1	Reload configuration (used by many services).
SIGINT	2	Interrupt from keyboard (Ctrl + C).
SIGTERM	15	Request graceful termination (default).
SIGKILL	9	Force kill (cannot be ignored or handled).
SIGSTOP	19	Stop (pause) the process (cannot be ignored).
SIGCONT	18	Continue a stopped process.

3. HOW SIGNALS WORK



4. GRACEFUL TERMINATION vs FORCE KILL

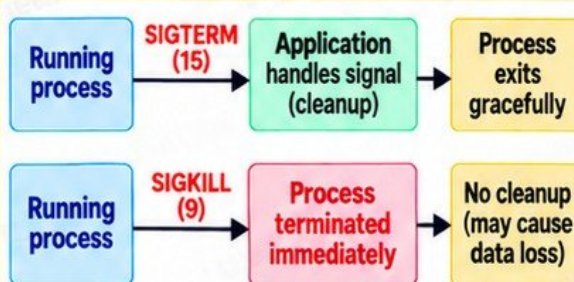
Graceful Termination (SIGTERM)

- ✓ Allows the application to clean up (open files, database connections, temporary files).
- ✓ Gives the process time to shut down correctly.
- ✓ Used by default (kill <pid> or kill -15).
- ✓ Preferred for services and production environments.

Force Kill (SIGKILL)

- ✗ Immediately terminates the process.
- ✗ Cannot be ignored or handled by the process.
- ✗ Does not allow cleanup.
- ✗ Can lead to data loss or corruption (e.g. incomplete writes).
- ✗ Should NOT be your first option. Use only when the process will not stop gracefully.

5. EXAMPLE: PROCESS LIFECYCLE WITH SIGNALS



6. USEFUL COMMANDS

```

# Send default termination signal (SIGTERM)
kill <pid>      # same as kill -15 <pid>

# Force kill (SIGKILL)
kill -9 <pid>

# Send a specific signal
kill -HUP <pid>  # reload configuration
kill -INT <pid>  # interrupt (Ctrl+C)
kill -STOP <pid> # pause process
kill -CONT <pid> # continue process

# Send signal by process name
pkill -TERM nginx
pkill -HUP nginx
  
```

7. VIEWING AND TRAPPING SIGNALS

View signal names and numbers

```

$ kill -l
1) SIGHUP    2) SIGINT    15) SIGTERM
9) SIGKILL   18) SIGCONT   19) SIGSTOP
  
```

See which signals a process handles

```

$ cat /proc/<pid>/status | grep Sig
SigQ:      0/515199
SigPnd:     0000000000000000
SigBlk:     0000000000000000
SigIgn:     0000000000000000
SigCgt:     000000000014002
  
```

Send signal by name using killall

```

$ killall -TERM java
  
```

8. REAL-WORLD EXAMPLES

- Reload Nginx configuration (without downtime)
`$ kill -HUP $(pgrep nginx)`
- Graceful shutdown of a Java application
`$ kill -TERM <pid>`
- Stop a stuck database process (force kill only if needed)
`$ kill -9 <pid>`
- Restart a service using systemctl (which sends appropriate signals)
`$ systemctl restart nginx`

9. COMMON SCENARIOS

- Application not responding → try SIGTERM first, then SIGKILL if needed.
- Reload configuration → use SIGHUP (e.g. Nginx, Apache).
- User pressed Ctrl + C → sends SIGINT.
- Process stuck in background → use SIGSTOP and SIGCONT.
- Service managed by systemd → use systemctl (handles signals correctly).

10. KEY TAKEAWAYS

- Signals are a safe and controlled way to manage processes.
- SIGTERM allows graceful termination and cleanup.
- SIGKILL should not be the automatic response.
- Different signals have different purposes (reload, interrupt, stop, continue).
- Use tools like kill, pkill, killall and systemctl.
- Always check why a process is not responding before force killing it.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Namespaces & cgroups



How namespaces provide isolation and cgroups control/account for resources. Connect both directly to Docker, Kubernetes and containerized workloads.

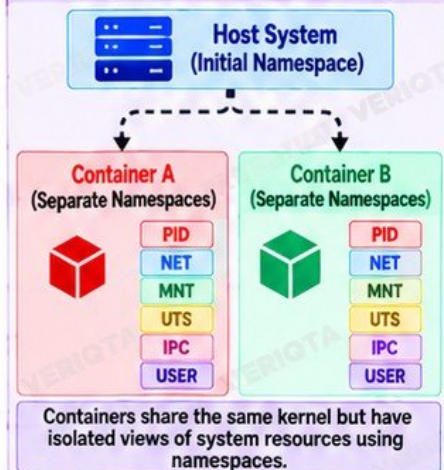
1. WHAT ARE NAMESPACES?

- Namespaces provide isolation by giving a process its own view of system resources.
- They make processes look like they are running on their own system (PID, network, filesystem, etc.).
- Processes in different namespaces cannot see each other's resources (depending on the namespace).
- Namespaces are used by containers (Docker, Kubernetes) to isolate workloads on the same host.

2. COMMON NAMESPACE TYPES

Namespace	What it isolates	Example use
PID	Process IDs	Container has its own process tree (PID 1).
NET	Network (interfaces, routing, ports)	Container has its own network stack.
MNT	Mount points (filesystem view)	Container sees its own filesystem.
UTS	Hostname and domain name	Container has its own hostname.
IPC	Inter-process communication	Isolates shared memory, semaphores, message queues.
USER	User and group IDs	Maps container root to non-root on host.
CGROUP	Cgroup root view	Restricts view of cgroups (used with cgroup v2).

3. HOW NAMESPACES WORK



4. WHAT ARE CGROUPS?

- cgroups (control groups) limit, account for and manage resource usage by a group of processes.
- They control CPU, memory, I/O, PIDs and more.
- cgroups do not provide isolation like namespaces, but they ensure fair resource sharing and prevent a container from consuming all host resources.
- Used by Docker and Kubernetes to enforce resource limits and track resource usage.

5. COMMON CGROUP CONTROLLERS

Controller	What it controls	Example use
cpu	CPU time and shares	Limit CPU usage (e.g. 0.5 CPU).
memory	Memory usage	Set memory limit (e.g. 512MiB).
io	Block I/O (disk)	Limit read/write I/O.
pids	Number of processes	Prevent fork bombs.
net_cls/net_prio	Network traffic class and priority	Control network usage (by class/priority).
cpuset	Specific CPU cores and NUMA nodes	Bind container to specific CPUs.

6. NAMESPACES vs CGROUPS

Aspect	Namespaces	cgroups
Purpose	Isolation	Resource control and accounting
What they manage	System resources view (PID, NET, MNT, UTS, IPC, USER)	CPU, memory, I/O, PIDs, network, etc.
Effect	Processes see separate view	Limit and track resource usage
Used by	Docker, Kubernetes (containers, pods)	Docker, Kubernetes (resource limits)
Works at	Kernel level	Kernel level
Relationship	Often used together in containers	Often used together in containers

7. PRACTICAL EXAMPLES

View namespaces of a process

```
$ ls -l /proc/$$/ns
lrwxrwxrwx 1 user user 0 Oct 2 10:00 pid -> pid:[4026531836]
lrwxrwxrwx 1 user user 0 Oct 2 10:00 net -> net:[4026531993]
lrwxrwxrwx 1 user user 0 Oct 2 10:00 mnt -> mnt:[4026531840]
lrwxrwxrwx 1 user user 0 Oct 2 10:00 uts -> uts:[4026531838]
...
```

View cgroup information (cgroup v2)

```
$ cat /sys/fs/cgroup/cgroup.controllers
cpu io memory pids ...
$ cat /sys/fs/cgroup/memory.max
536870912 # 512MiB limit
```

Run a process in a new namespace

```
$ unshare -n bash
$ ip addr # shows isolated network namespace
```

Check container namespaces (Docker)

```
$ docker inspect <container> --format '{{.State.Pid}}'
$ ls -l /proc/<pid>/ns
```

Check cgroup usage

```
$ cat /sys/fs/cgroup/memory.current
134217728 # current memory usage (128MiB)
$ cat /sys/fs/cgroup/cpu.max
100000 200000 # CPU quota and period
```

8. CONTAINER REAL-WORLD CONNECTION



- Uses namespaces to isolate containers.
- Uses cgroups to enforce resource limits (CPU, memory, etc.).
- Each container runs as a set of namespaces + cgroups.



- Each pod gets its own namespaces (e.g. network, PID).
- Uses cgroups to enforce resource requests and limits.
- Helps multiple containers run safely on the same node without interfering.

9. KEY TAKEAWAYS

- Namespaces provide isolation by giving processes their own view of system resources.
- cgroups control, limit and account for resource usage.
- Both are used together in Docker and Kubernetes.
- Namespaces isolate, cgroups manage resources.
- Understanding both helps you troubleshoot container behaviour and resource issues.

10. COMMON TROUBLESHOOTING SCENARIOS

- Container cannot see host processes → different PID namespace.
- Application cannot reach the network → check network namespace.
- Container using too much memory → check cgroup memory limit/usage.
- Container slow or high CPU usage → check cgroup CPU limits.
- Too many processes in container → check pids cgroup limit.
- Unexpected filesystem view → check mount namespace.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Linux Services & systemd

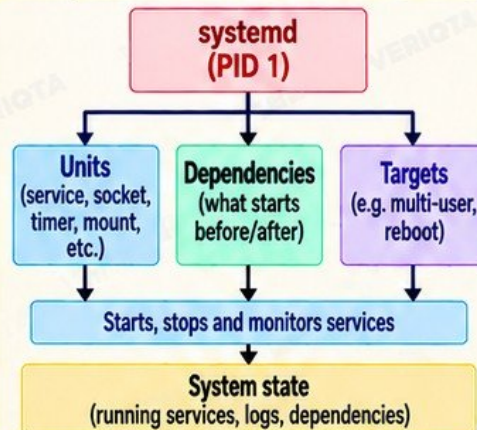


PID 1, units, dependencies, service lifecycle, logs and failure states.
Troubleshoot a failed service using systemctl and journalctl.

1. WHAT IS systemd?

- systemd is the init system and service manager used by most modern Linux distributions.
- It is PID 1 (the first process) and manages the entire system.
- It starts services, manages dependencies, handles failures and provides logging.
- It uses units to represent services, sockets, timers, mounts and more.

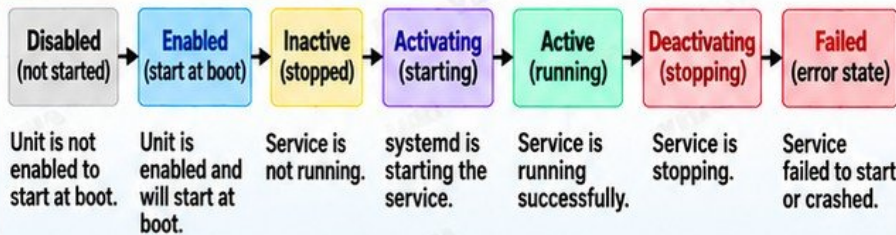
2. systemd ARCHITECTURE (PID 1)



3. COMMON UNIT TYPES

Unit Type	Purpose	Example
service	Manages a long running process.	nginx.service
socket	Listens on a socket and activates service.	docker.socket
target	Group of units (system state).	multi-user.target
timer	Runs tasks on a schedule.	backup.timer
mount	Manages filesystem mounts.	data.mount
path	Watches for file changes.	app.path
slice	Groups processes for resource control.	system.slice

4. SERVICE LIFECYCLE



5. UNIT DEPENDENCIES

After=	Starts this unit after another unit.
Before=	Starts this unit before another unit.
Requires=	Unit must be started (fails if not).
Wants=	Unit is desirable but not required.
Conflicts=	Units cannot run at the same time.
PartOf=	Ties lifecycle to another unit.
WantedBy=	Defines which target enables this unit.

6. COMMON systemctl COMMANDS

Command	Purpose
systemctl status <service>	Show service status.
systemctl start <service>	Start a service.
systemctl stop <service>	Stop a service.
systemctl restart <service>	Restart a service.
systemctl enable <service>	Enable at boot.
systemctl disable <service>	Disable at boot.
systemctl is-enabled <service>	Check if enabled.
systemctl list-units --type=service	List all services.
systemctl list-dependencies <service>	Show dependencies.

7. VIEWING LOGS WITH journalctl

View logs for a service
<code>\$ journalctl -u nginx.service</code>
Follow logs in real time
<code>\$ journalctl -u nginx.service -f</code>
View recent logs (last 50 lines)
<code>\$ journalctl -u nginx.service -n 50</code>
View logs since a specific time
<code>\$ journalctl -u nginx.service --since "2024-10-01 10:00"</code>
View only error logs
<code>\$ journalctl -u nginx.service -p err</code>

8. TROUBLESHOOTING A FAILED SERVICE

1	Check service status <code>\$ systemctl status <service></code>
2	Read recent logs <code>\$ journalctl -u <service> -n 50</code>
3	Identify the failure reason Look for error messages in the logs.
4	Check dependencies <code>\$ systemctl list-dependencies <service></code>
5	Check configuration Validate the service configuration file.
6	Fix the issue and restart <code>\$ systemctl restart <service></code>
7	Verify it is running <code>\$ systemctl status <service></code>

9. EXAMPLE: TROUBLESHOOT A FAILED NGINX SERVICE

```

$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled)
   Active: failed (Result: exit-code) since Thu 2024-10-01 10:15:23
   Process: 1234 ExecStart=/usr/sbin/nginx (code=exited, status=1/FAILURE)

$ journalctl -u nginx -n 5
Oct 01 10:15:23 server nginx[1234]: nginx: [emerg] bind() to 0.0.0.0:80
Oct 01 10:15:23 server nginx[1234]: failed (98: Address already in use)
Oct 01 10:15:23 server systemd[1]: nginx.service: Failed with result
Oct 01 10:15:23 server systemd[1]: Failed to start A high performance web server.
  
```

10. COMMON FAILURE STATES

State	Meaning
active (running)	Service is running.
inactive (dead)	Service is stopped.
activating	Service is starting.
deactivating	Service is stopping.
failed	Service failed to start or crashed.
reloaded	Service configuration reloaded.

11. KEY TAKEAWAYS

- systemd is PID 1 and manages the system and all services.
- Services are managed as units with clear dependencies.
- Use systemctl to manage services and journalctl to view logs.
- Always check logs to understand why a service failed.
- Do not just restart — identify and fix the root cause.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Troubleshooting Linux from Evidence



Final production scenario: application is slow. Work through CPU → memory → process → disk → network → logs → kernel evidence → hypothesis → fix → verification.

1. SCENARIO



A user reports:

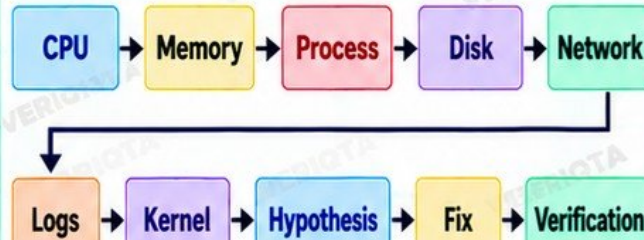
- The application is slow
- Response time is high
- Users are experiencing timeouts



GOAL

Find the real cause using evidence, not guesses, and restore normal performance.

2. TROUBLESHOOTING FLOW



3. KEY PRINCIPLES

- Start with observable symptoms.
- Use real evidence from the system.
- Check each layer in a logical order.
- Form a hypothesis based on evidence.
- Apply a targeted fix.
- Verify that the issue is resolved.
- Avoid making changes without understanding the cause.

4. CHECK CPU



Is the CPU under high usage?

```
$ top
$ htop
$ mpstat 1
$ sar -u 1 5
```

What to look for

- High %CPU (user, system or iowait).
- Identify the process using CPU.
- Check if the load is sustained.

5. CHECK MEMORY



Is the system under memory pressure?

```
$ free -h
$ vmstat 1
$ sar -r 1 5
$ cat /proc/meminfo
```

What to look for

- Low available memory.
- High swap usage.
- Processes using large memory.
- OOM kills in logs.

6. CHECK PROCESS



Which process is causing the issue?

```
$ ps aux --sort=-%cpu
$ ps aux --sort=-%mem
$ top -p <pid>
$ pstree -p <pid>
```

What to look for

- Identify the exact process.
- Check process age and owner.
- Confirm it is part of the application.
- See if it is making progress.

7. CHECK DISK



Is there a disk performance issue?

```
$ iostat -x 1
$ df -h
$ du -sh <dir>
$ lsblk
$ dmesg | grep -i error
```

What to look for

- High disk I/O and high iowait.
- Disk latency and throughput.
- Disk space usage.
- Filesystem or disk errors.

8. CHECK NETWORK



Is there a network bottleneck?

```
$ ss -tulnp
$ ss -tnp
$ ip addr show
$ ip route
$ sar -n DEV 1 5
```

What to look for

- Listening ports and connections.
- High network traffic or drops.
- Network errors (retransmits, drops).
- Interface issues or routing problems.

9. CHECK LOGS



What do the application and system logs show?

```
$ journalctl -u <service> -n 50
$ journalctl -xe
$ tail -f /var/log/<app>.log
$ grep -i error /var/log/syslog
```

What to look for

- Application errors or exceptions.
- Service failures or restarts.
- Permission errors.
- Any clues around the time it became slow.

10. CHECK KERNEL EVIDENCE



Any kernel-level issues?

```
$ dmesg | tail -n 50
$ dmesg | grep -iE 'error|fail'
$ cat /proc/uptime
$ cat /proc/pressure/*
$ sar -d 1 5
```

What to look for

- Hardware or driver errors.
- Disk or filesystem issues.
- OOM killer activity.
- Resource pressure (CPU, memory, I/O).

11. FORM A HYPOTHESIS



Based on the evidence, what is likely causing the slowness?

Examples of hypotheses

- A process is consuming excessive CPU.
- The application is hitting memory limits.
- Disk I/O is slow due to high latency.
- The application is waiting on a database or external service.
- There is network latency or packet loss.
- The application is experiencing errors and retrying.
- The system is under kernel resource pressure.

12. APPLY A TARGETED FIX



```
$ systemctl restart <service>
$ kill -TERM <pid> # graceful
$ renice -n 10 <pid> # reduce CPU priority
$ systemctl enable <service>
$ <application specific fix>
```

Examples of fixes

- Restart a service or application.
- Fix misconfiguration.
- Free up disk space.
- Optimise a slow query or external dependency.
- Adjust resource limits (CPU, memory, I/O).

13. VERIFY THE RESOLUTION



```
$ top
$ free -h
$ iostat -x 1
$ ss -tnp
$ curl -I http://localhost # test response
$ journalctl -u <service> -f
```

Verify that

- CPU, memory, disk and network are normal.
- The application is responding quickly.
- No new errors in logs.
- Users can access the service successfully.

14. COMMON TOOLS REFERENCE



Area	Useful commands
CPU	top, htop, mpstat, sar -u
Memory	free, vmstat, sar -r, /proc/meminfo
Process	ps, top -p, pstree, strace
Disk	iostat, df, du, lsblk
Network	ss, ip, sar -n, ping, traceroute
Logs	journalctl, tail, grep
Kernel	dmesg, /proc/pressure/*, sar
Service	systemctl, journalctl -u
Test	curl, wget, nc, ping, time



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta